

Action Plan: Computational Complexity

ICLR 2027 SUBMISSION PREP – ROBUST ACTIVATION STEERING

Objective: To preemptively neutralize reviewer critiques regarding the computational overhead of the H_∞ robust control algorithm. By acquiring hard empirical data and documenting identical inference-time latency compared to nominal A-LQR, we transform a potential theoretical vulnerability into a rigorous core selling point.

Phase 1: Empirical Benchmarking (Lab Tasks)

Assign the following direct measurement tasks to generate the raw data required for the manuscript updates.

Task 1: Offline Synthesis Profiling

Profile the calibration phase for one representative model (e.g., Llama-3-8B) and one behavior (e.g., HarmBench refusal). Measure the exact wall-clock time in seconds for A-LQR synthesis versus H_∞ synthesis. Ensure all caches are cleared before timing.

Task 2: Bisection Iteration Tracking

Modify the synthesis script to log the average number of bisection iterations (N_{iter}) required for Algorithm 4 to converge to γ^* . Calculate the mean and maximum N_{iter} across a sample of 100 calibration prompts. This justifies the theoretical scaling multiplier in the paper.

Task 3: Inference Latency Measurement

Run a generation batch (e.g., 50 tokens) using the base model, the A-LQR deployed controller, and the H_∞ deployed controller. Record the average milliseconds per generated token and peak VRAM allocation. The goal is to empirically prove the deployment overhead is strictly identical.

Task 4: Hardware & Software Documentation

Record the exact hardware stack used for these benchmarks (e.g., single NVIDIA A100 80GB, CUDA version, PyTorch version). This information must be logged precisely for the reproducibility appendix.

Phase 2: Manuscript Integration (Writing Tasks)

Once the benchmarking data is collected, implement the following structural updates in the LaTeX source.

1. Main Text Updates (Results / Discussion Section)

Insert a compact summary table immediately following the primary performance results (e.g., after the L-CiteEval block). Precede the table with this bridging paragraph:

"While H_∞ explicitly optimizes for worst-case disturbance attenuation, its reliance on a reduced-state backward Riccati recursion results in computational complexity practically identical to nominal A-LQR (Table X). Complete complexity bounds and empirical runtimes are detailed in Appendix M."

Table Format to Implement:

Method	Synthesis Complexity	Synthesis Time (s)	Inference Overhead (ms/token)	Peak Memory (GB)
A-LQR	$O(L \cdot r_x^3)$	[Data from Task 1]	[Data from Task 3]	[Data from Task 3]
H_∞ (Ours)	$O(N_{iter} \cdot L \cdot r_x^3)$	[Data from Task 1]	[Data from Task 3]	[Data from Task 3]

2. New Appendix: Computational Complexity (Appendix M)

- M.1 Asymptotic Synthesis Complexity:** State that computing M_k , G_k , and F_k in Algorithm 4 requires operations on $r_x \times r_x$ matrices, yielding a per-iteration complexity of $O(L \cdot r_x^3)$. Multiply by N_{iter} for the final bound.
- M.2 Empirical Synthesis Runtime:** Report the data from Lab Tasks 1, 2, and 4. Detail the hardware specs.
- M.3 Inference-Time Deployment:** State explicitly that the deployed controller requires exactly one matrix-vector multiplication per layer, yielding an $O(L \cdot d \cdot r_x)$ inference complexity—mathematically identical to A-LQR.

Phase 3: Reviewer Defense Matrix

Ensure the manuscript directly maps these empirical findings to standard reviewer concerns.

Potential Reviewer Critique	Pre-emptive Countermeasure in Draft
"The dynamic game backward recursion is too slow to scale to large language models."	Point to Table X in the main text showing the offline synthesis time difference is nominal (measured in seconds), as both methods rely on standard Riccati recursions.
"Does this state-aware approach slow	Highlight Appendix M.3, mathematically proving that the deployed